

AI FAQ ASSISTANT

AI-POWERED WEB APPLICATION FOR INTELLIGENT FAQ SEARCH AND ACADEMIC INFORMATION SUPPORT

PROJECT DOCUMENTATION

Submitted By:

Team ID : SWTID-2026-9997

Team Leader : Shalini K

Team member : Sindhuja S

Team member : Sabrin Nisha K

Team member : Yamini A

Team member : Sankari N

Course: B.Sc. Computer Science with Artificial Intelligence

College: DRBCCC Hindu College

University: University of Madras

Academic Year: 2026-2027

1. ABSTRACT

The AI FAQ Assistant is a web-based academic support application developed to help students quickly find answers to frequently asked questions related to college activities, courses, examinations, fees, admissions, placements, scholarships, and technical support. The system provides a centralized platform where users can search and access frequently asked questions through an easy-to-use dashboard.

The application is developed using Node.js and Express.js for the backend, MongoDB with Mongoose for database management, and HTML, CSS, and JavaScript for the frontend. User authentication is implemented using JWT (JSON Web Token), while passwords are securely stored using bcryptjs. The system supports separate user and administrator functionalities, allowing administrators to manage users, categories, and FAQs.

A custom AI query processing engine is integrated into the application to understand user questions and identify the most relevant FAQ based on keyword and text similarity. The system calculates a matching score and confidence level and can also provide related FAQ suggestions. If a suitable answer is not available, the system informs the user and suggests relevant categories.

The main objective of the project is to reduce the time required to search for common academic information and provide students with a simple, organized, and interactive FAQ assistance system. The application demonstrates the practical integration of web development, database management, authentication, and intelligent query processing in an academic environment.

2. INTRODUCTION

In an academic environment, students frequently need information about courses, examinations, admissions, fees, placements, scholarships, college-related activities, and technical support. Finding answers to these common questions manually can take time, especially when the required information is distributed across different sources.

The AI FAQ Assistant is developed as a web-based application to provide a centralized platform for accessing frequently asked questions and their answers. The system allows users to browse FAQs by category, search for specific questions, and interact with an AI-assisted query system when they need a more direct answer.

The application uses Node.js and Express.js to provide the backend services and MongoDB with Mongoose for storing users, categories, and FAQ information. The frontend is implemented using HTML, CSS, and JavaScript, providing pages for registration, login, dashboard access, FAQ browsing, and AI-assisted queries.

The system also provides authentication using JSON Web Tokens (JWT) and password protection using bcryptjs. Administrator functionality allows authorized administrators to manage users, categories, and FAQ records. The custom AI query processing engine compares the user's question with available FAQ information using keyword and text similarity and returns the most relevant result when a suitable match is found.

The main purpose of the project is to make academic information easier to access through a simple, organized, and interactive web application. It combines web application development, database management, authentication, and intelligent FAQ query processing into a single academic support system.

3. PROJECT TITLE

AI FAQ ASSISTANT

AI-POWERED WEB APPLICATION FOR INTELLIGENT FAQ SEARCH AND ACADEMIC INFORMATION SUPPORT

4. PROJECT DESCRIPTION

The AI FAQ Assistant is a web-based application developed to provide students with a

centralized platform for accessing frequently asked questions and academic information. The application combines a structured FAQ management system with a custom AI query processing engine to help users find relevant answers more easily.

The system is built using Node.js and Express.js for server-side application development. MongoDB is used as the database, with Mongoose providing schema definition and database interaction. The frontend consists of HTML, CSS, and JavaScript pages that allow users to register, log in, browse FAQs, search for information, and use the AI Assistant.

The application organizes FAQs into categories such as General, College, Courses, Exams, Fees, Admission, Placements, Scholarships, and Technical Support. Users can search FAQs using keywords and filter them based on categories. The dashboard also provides recently viewed FAQ information for easier access.

The AI Assistant uses a custom keyword and text similarity-based query processing engine. When a user enters a question, the system processes the query, compares it with the available FAQ questions, keywords, and answers, and calculates a matching score. When a suitable match is found, the system returns the relevant FAQ along with a confidence value and related FAQ suggestions.

The application also provides secure user authentication using JWT, while passwords are protected using bcryptjs. Administrators can manage users, FAQ records, and categories through protected administrative functions. The backend follows a modular structure using routes, controllers, models, middleware, and utility modules to organize the application functionality.

Overall, the project provides an integrated platform for FAQ management, academic information retrieval, user authentication, category-based organization, and intelligent question matching within a single web application.

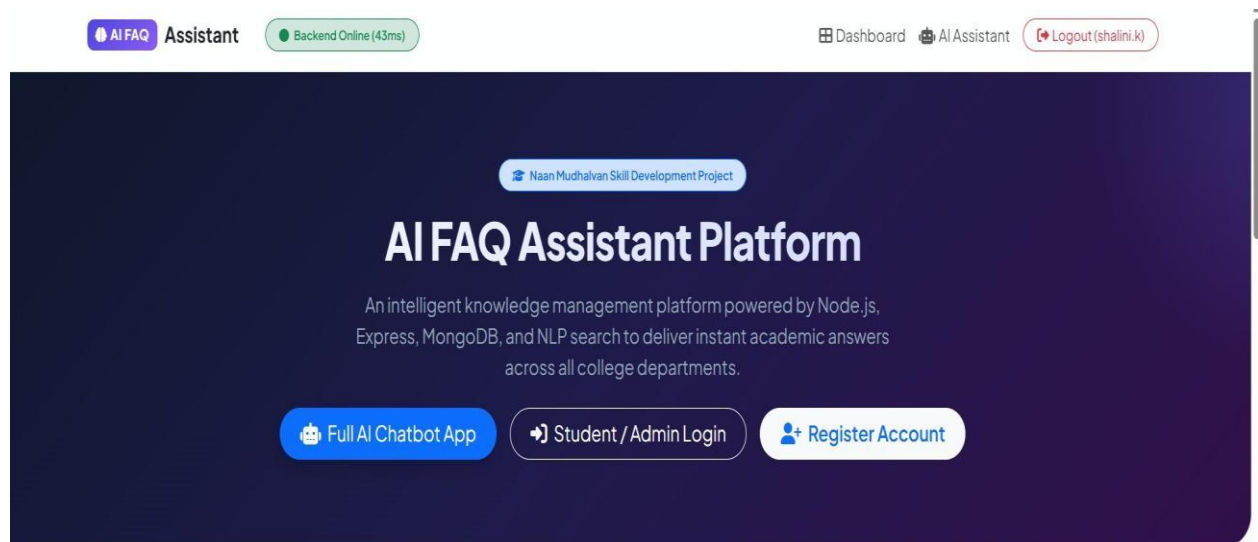


Figure 1: AI FAQ Assistant Home Page

5. PROBLEM STATEMENT

Students regularly require information related to courses, examinations, admissions, fees, placements, scholarships, college activities, and technical support. When such information is not available through a single organized platform, students may spend additional time searching through different sources or repeatedly asking for the same information.

The major problems addressed by the AI FAQ Assistant are:

5.1 Difficulty in Finding Academic Information

Students may need to search for information related to different academic and college activities. Without a centralized FAQ system, finding the required information can become time-consuming.

5.2 Repeated Questions

Many students have similar or repeated questions about courses, exams, fees, admissions, placements, and other college-related services. Handling these questions individually can result in repeated effort.

5.3 Unorganized FAQ Information

Frequently asked questions need to be properly categorized and maintained. Without categorybased organization, users may find it difficult to locate relevant information quickly.

5.4 Limited Intelligent Query Assistance

A normal FAQ search may depend mainly on matching exact words or phrases. Users may enter a question in a different form and still expect the system to identify the relevant FAQ.

5.5 Manual FAQ Management

FAQ information needs to be created, updated, and removed when required. A proper administrative system is required to manage FAQ records and categories efficiently.

5.6 Need for Secure User Access

User accounts and administrative operations require authentication and authorization. Without proper access control, protected operations such as managing users, FAQs, and categories may not be secure.

6. PROPOSED SOLUTION

The proposed AI FAQ Assistant provides a centralized web-based platform for managing and accessing frequently asked questions. The system is designed to reduce the time required to find common academic information by organizing FAQs into clearly defined categories and providing an intelligent question-matching facility.

The application provides a user-friendly interface through which users can register, log in, browse FAQ categories, search for questions, and interact with the AI Assistant. FAQs are stored in a MongoDB database and are accessed through RESTful API endpoints developed using Node.js and Express.js.

The system uses a custom query-processing engine instead of relying on an external AI service. When a user submits a question, the engine processes the input and compares it with the available FAQ data using keyword and text similarity. It calculates a matching score and identifies the most relevant FAQ when the score reaches the required threshold. Related FAQ suggestions can also be returned to help users find additional information.

For administrative management, protected API operations allow authorized administrators to manage users, categories, and FAQ records. JWT-based authentication is used to protect restricted operations, while bcryptjs is used for password hashing.

The proposed solution therefore integrates FAQ management, category-based organization, secure authentication, database management, and intelligent query matching into a single academic information support application.

7. OBJECTIVES

The main objective of the AI FAQ Assistant is to develop a centralized web application that helps users access frequently asked academic information quickly and efficiently. The system combines FAQ management, category-based organization, secure authentication, and intelligent question matching.

7.1 Primary Objectives

- To develop a centralized platform for storing and accessing frequently asked questions.
- To provide users with an easy-to-use interface for browsing, searching, and filtering FAQs.
- To implement an intelligent query-processing mechanism that identifies relevant FAQ answers based on the user's question.
- To organize FAQ information into categories such as Courses, Exams, Fees, Admission, Placements, Scholarships, and Technical Support.
- To provide secure authentication and authorization using JWT-based access control.
- To provide administrators with functions for managing users, FAQs, and categories.

7.2 Secondary Objectives

- To store and manage application data using MongoDB and Mongoose.
- To protect user passwords using bcryptjs hashing.
- To provide confidence information and related FAQ suggestions for matched AI queries.
- To maintain a modular backend structure using routes, controllers, models, middleware, and utility modules.
- To provide centralized error handling for reliable application operation.
- To create a scalable foundation that can be extended with additional FAQ categories and intelligent assistance features in the future.

8. SYSTEM REQUIREMENTS

The AI FAQ Assistant requires a suitable software environment and basic computer hardware to run the frontend, backend server, and MongoDB database successfully.

8.1 Software Requirements

- **Operating System:** Windows 10/11 or another operating system capable of running Node.js and MongoDB.
- **Node.js:** Used as the JavaScript runtime environment for executing the backend server.
- **npm (Node Package Manager):** Used to install and manage the dependencies required by the project.
- **Express.js:** Used as the backend web framework for creating RESTful API routes and handling HTTP requests.
- **MongoDB:** Used as the primary database for storing user, category, and FAQ information.
- **Mongoose:** Used as the Object Data Modeling (ODM) library for defining schemas and interacting with MongoDB.
- **Visual Studio Code:** Used as the development environment for creating and maintaining the project files.
- **Web Browser:** Required to access and interact with the frontend application.
- **Postman:** Used for testing and verifying the REST API endpoints during development.

8.2 Hardware Requirements

The project requires a computer capable of running the Node.js server, database service, code editor, and web browser simultaneously.

- **Processor:** A modern Intel or AMD processor.
- **RAM:** A minimum of 4 GB RAM is suitable for basic project execution.
- **Storage:** Sufficient free storage for the project files, Node.js packages, MongoDB data, and development tools.
- **Display:** A standard monitor or laptop display suitable for using the development environment and web browser.
- **Network:** An internet connection may be required for initial package installation and development-related resources.

9. TECHNOLOGY USED

The AI FAQ Assistant is developed using a combination of backend, frontend, database, authentication, and query-processing technologies. Each technology is used for a specific purpose in the development and operation of the application.

9.1 Node.js

Node.js is used as the JavaScript runtime environment for executing the backend application. It handles server-side operations and provides the environment required to run the Express.js application.

9.2 Express.js

Express.js is used as the backend web framework. It provides routing and request-handling functionality for the application's REST API endpoints and connects the frontend requests with the appropriate controllers.

9.3 MongoDB

MongoDB is used as the primary database for storing application information. The project stores data related to users, categories, and frequently asked questions in MongoDB collections.

9.4 Mongoose

Mongoose is used as the Object Data Modeling (ODM) library for MongoDB. It defines the data schemas for the User, Category, and FAQ models and provides methods for database operations and validation.

9.5 JWT

JSON Web Token (JWT) is used for authentication and protected access. After successful login, a token is generated and used to authenticate requests to protected API routes.

9.6 bcryptjs

bcryptjs is used to securely hash user passwords before they are stored in the database. During login, the entered password is compared with the stored hashed password.

9.7 HTML, CSS and JavaScript

HTML, CSS, and JavaScript are used to develop the frontend interface. HTML provides the page structure, CSS manages the appearance and layout, and JavaScript handles user interactions and communication with the backend API.

9.8 Custom AI Query Processing Engine

The project uses a custom AI query-processing engine developed within the application. It processes user questions using tokenization, keyword matching, question matching, answer matching, and similarity scoring to identify the most relevant FAQ.

9.9 REST API

The application uses RESTful API endpoints to connect the frontend with the backend. Separate API routes are provided for authentication, categories, FAQs, AI queries, and user management.

9.10 Postman

Postman is used for testing and verifying API endpoints during development. It can be used to send HTTP requests and inspect the responses returned by the backend.

10. SYSTEM ARCHITECTURE

The AI FAQ Assistant follows a modular client-server architecture in which the frontend, backend API, authentication middleware, database, and custom AI query-processing engine work together to provide academic FAQ assistance.

10.1 Frontend Layer

The frontend is developed using HTML, CSS, and JavaScript. It provides the user interface for registration, login, FAQ browsing, searching, dashboard access, AI assistance, and administrative operations.

10.2 Express Server Layer

The backend application runs on Node.js with Express.js. It receives HTTP requests from the frontend, maps requests to the appropriate API routes, processes the required operations, and returns responses to the client.

10.3 Authentication and Authorization Layer

The authentication middleware verifies JWT tokens included in protected requests. It identifies authenticated users and applies administrator-level access control to restricted operations.

10.4 Controller Layer

Controllers contain the main application logic for different modules:

- **Authentication Controller** – handles registration, login, and user information.
- **FAQ Controller** – handles FAQ retrieval, creation, updating, deletion, searching, and filtering.
- **Category Controller** – manages FAQ categories.
- **User Controller** – provides administrative user management and statistics.
- **AI Controller** – processes user questions through the AI query engine.

10.5 Database Layer

The application uses MongoDB as its database and Mongoose for database interaction. The main database entities are:

- **User**
- **Category**
- **FAQ**

These models define the structure and validation rules for application data.

10.6 AI Query Processing Layer

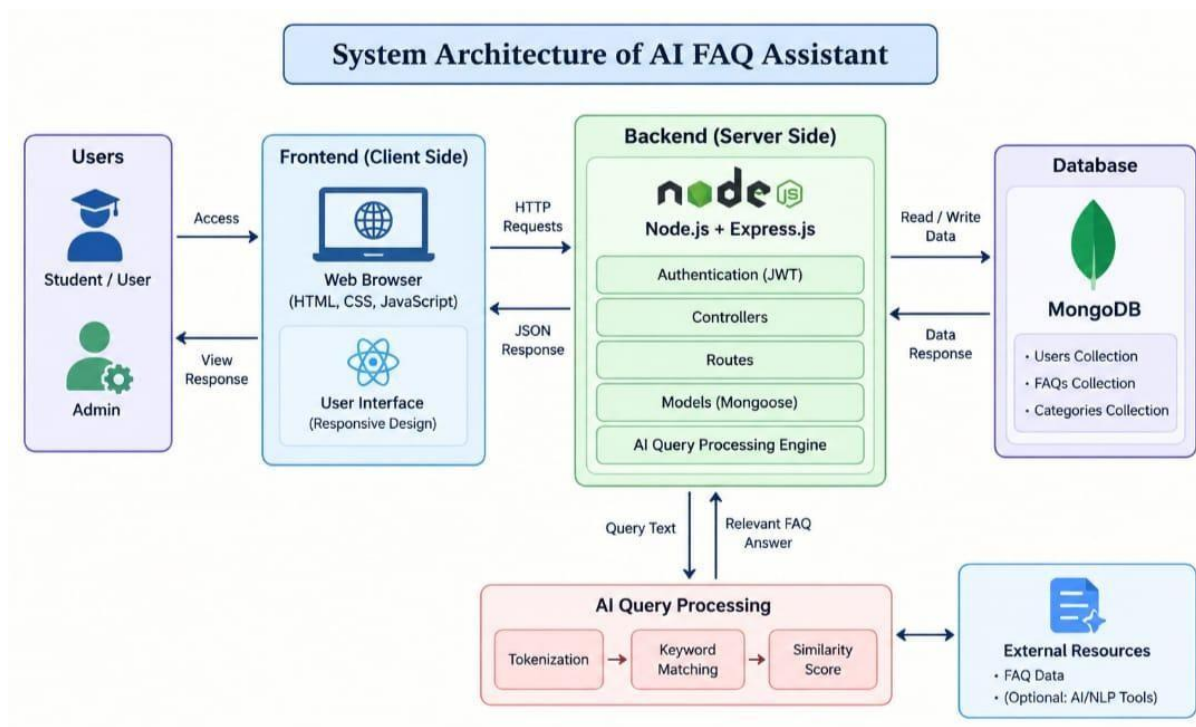
The custom AI query-processing engine receives the user's question and compares it with the available FAQ information. It performs tokenization and similarity calculations using FAQ questions, keywords, and answers to determine the most relevant result.

10.7 Request Flow

The overall system request flow can be represented as:

User → Frontend → Express.js API → Routes → Middleware/Controller → MongoDB / AI Engine → Response → Frontend

This architecture separates the presentation, application logic, authentication, data management, and query-processing responsibilities, allowing the different components of the AI FAQ Assistant to work together in an organized manner.



11. MVC ARCHITECTURE

The AI FAQ Assistant follows a modular Model-View-Controller (MVC) architecture to separate data management, application processing, and user interface responsibilities. This structure makes the project easier to organize, maintain, test, and extend.

11.1 Model Layer

The Model Layer is responsible for defining and managing the application's data structure. The project uses Mongoose to define MongoDB schemas and perform database operations.

The main models are:

- **User Model:** Stores user name, email, password, role, and creation information.
- **Category Model:** Stores category name, description, and creation information.
- **FAQ Model:** Stores questions, answers, category reference, keywords, views, and timestamps.

The models also provide validation rules to maintain the structure and consistency of stored data.

11.2 Controller Layer

The Controller Layer contains the main application logic and processes requests received from the API routes.

The project includes the following controllers:

- **Auth Controller** – handles user registration, login, and authenticated user information.
- **FAQ Controller** – handles FAQ creation, retrieval, searching, updating, and deletion.
- **Category Controller** – handles category creation, retrieval, updating, deletion, and FAQ counts.
- **User Controller** – handles administrator user management, user statistics, role updates, and user deletion.
- **AI Controller** – receives user questions and sends them to the custom AI queryprocessing engine.

11.3 Routing Layer

The Routing Layer defines the API endpoints through which the frontend communicates with the backend.

The major route groups are:

- `/api/auth` – authentication operations
- `/api/categories` – category operations
- `/api/faqs` – FAQ operations
- `/api/ai` – AI Assistant queries
- `/api/users` – user management operations

The routes connect incoming HTTP requests to the corresponding controller functions.

11.4 Middleware Layer

The middleware layer processes requests before they reach protected controller functions.

The project includes:

- **Authentication Middleware:** Verifies JWT tokens and protects authenticated routes.
- **Admin Authorization:** Restricts administrator operations to users with the admin role.
 - **Error Middleware:** Provides centralized handling of application errors.

11.5 AI Processing Layer

The project also contains a custom AI processing utility that works with the controller layer. The AI Engine processes the user's question, compares it with available FAQ information, calculates similarity scores, and returns the most relevant result.

11.6 MVC Request Flow

The general MVC flow of the application is:

User → View → Route → Middleware → Controller → Model / AI Engine → Controller → Response → View

This separation allows each part of the application to perform a specific responsibility while working together as a complete AI FAQ Assistant system.

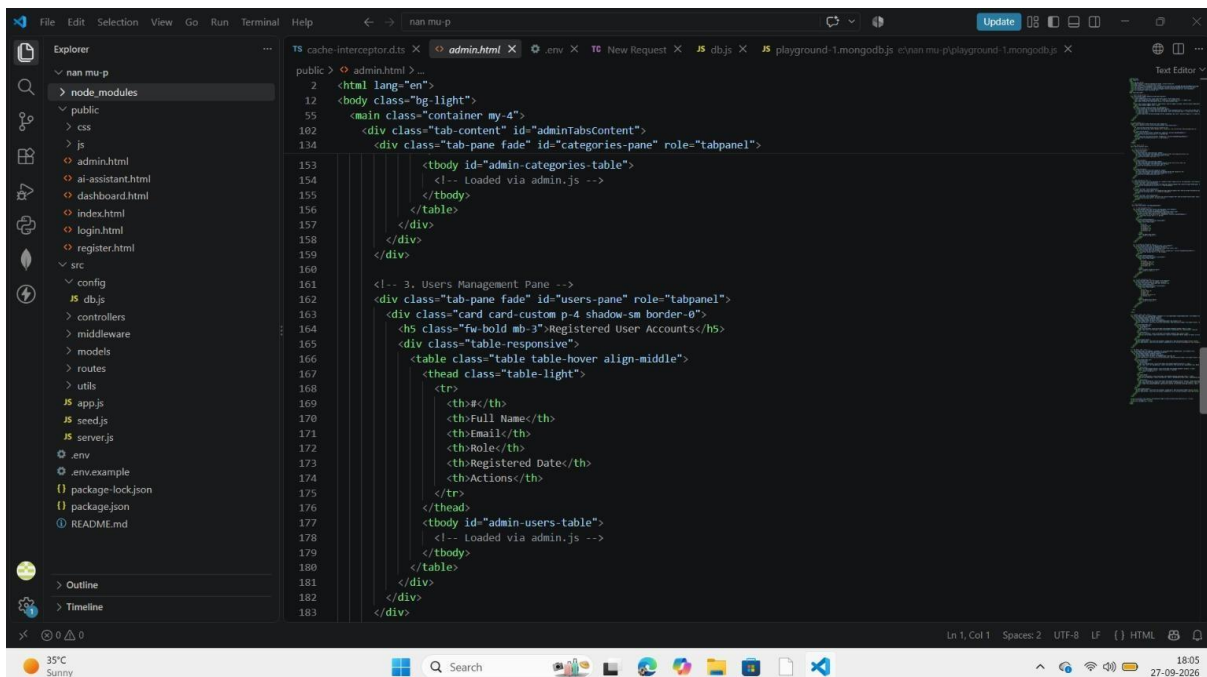


Figure 2: Project Structure and Source Code

JS authController.js X

nan mu-p > src > controllers > JS authController.js > ...

```
1 const User = require('../models/User');
2 const bcrypt = require('bcryptjs');
3 const jwt = require('jsonwebtoken');
4
5 // Helper to generate JWT token
6 const generateToken = (id) => {
7   return jwt.sign(
8     { id },
9     process.env.JWT_SECRET || 'naan_mudhalvan_ai_faq_secret_key_2026_super_secure',
10    { expiresIn: '30d' }
11  );
12 };
13
14 // @desc   Register a new user
15 // @route   POST /api/auth/register
16 // @access  Public
17 const register = async (req, res, next) => {
18   try {
19     const { name, email, password, confirmPassword, role } = req.body;
20
21     // Validate inputs
22     if (!name || !email || !password) {
23       return res.status(400).json({
24         success: false,
25         message: 'Please provide name, email, and password'
26       });
27     }
28
29     if (confirmPassword && password !== confirmPassword) {
30       return res.status(400).json({
31         success: false,
32         message: 'Passwords do not match'
33       });
34     }
35
36     if (password.length < 6) {
37       return res.status(400).json({
38         success: false,
39         message: 'Password must be at least 6 characters long'
40       });
41     }
42
43     // Check duplicate email
44     const existingUser = await User.findOne({ email: email.toLowerCase() });
45     if (existingUser) {
46       return res.status(400).json({
47         success: false,
48         message: 'Email address is already registered'
49       });
50     }
51
52     // Hash password securely with bcrypt
53     const salt = await bcrypt.genSalt(10);
54     const hashedPassword = await bcrypt.hash(password, salt);
55
56     // Create user
57     const user = await User.create({
58       name,
59       email: email.toLowerCase(),
60       password: hashedPassword,
61       role: role && ['admin', 'user'].includes(role) ? role : 'user'
62     });
63
64     // Generate Token
65     const token = generateToken(user._id);
66
67     res.status(201).json({
68       success: true,
69       message: 'User registered successfully',
70       token
71     });
72   } catch (error) {
73     next(error);
74   }
75 }
```

```

67     res.status(201).json({
68       success: true,
69       message: 'User registered successfully',
70       token,
71       user: {
72         id: user._id,
73         name: user.name,
74         email: user.email,
75         role: user.role,
76         createdAt: user.createdAt
77       }
78     });
79   } catch (error) {
80     next(error);
81   }
82 };
83
84 // @desc   Authenticate user & get token
85 // @route   POST /api/auth/login
86 // @access  Public
87 const login = async (req, res, next) => {
88   try {
89     const { email, password } = req.body;
90
91     if (!email || !password) {
92       return res.status(400).json({
93         success: false,
94         message: 'Please enter both email and password'
95       });
96     }
97
98     // Check user & select password explicitly
99     const user = await User.findOne({ email: email.toLowerCase() }).select('+password');
100
101     if (!user) {
102       return res.status(401).json({
103         success: false,
104         message: 'Invalid email or password'
105       });
106     }
107
108     // Verify password with bcrypt
109     const isMatch = await bcrypt.compare(password, user.password);
110     if (!isMatch) {
111       return res.status(401).json({
112         success: false,
113         message: 'Invalid email or password'
114       });
115     }
116
117     // Generate token
118     const token = generateToken(user._id);
119
120     res.status(200).json({
121       success: true,
122       message: 'Login successful',
123       token,
124       user: {
125         id: user._id,
126         name: user.name,
127         email: user.email,
128         role: user.role,
129         createdAt: user.createdAt
130       }
131     });
132   } catch (error) {
133     next(error);
134   }
135 };
136
137 // @desc   Get current user profile
138 // @route   GET /api/auth/me
139 // @access  Private
140 const getMe = async (req, res, next) => {
141   try {
142     const user = await User.findById(req.user.id);
143     res.status(200).json({
144       success: true,
145       user: {
146         id: user._id,
147         name: user.name,
148         email: user.email,
149         role: user.role,
150         createdAt: user.createdAt
151       }
152     });
153   } catch (error) {
154     next(error);
155   }
156 };
157
158 module.exports = {
159   register,
160   login,
161   getMe
162 };
163

```

Figure 3: Project Structure and Source Code

12. DATABASE DESIGN

The AI FAQ Assistant uses MongoDB as its primary database for storing and managing application data. Mongoose is used as the ODM (Object Data Modeling) library to define schemas, apply validation rules, and perform database operations.

The database is organized into three main collections:

- **Users**
- **Categories**
- **FAQs**

These collections support user authentication, FAQ organization, FAQ retrieval, and administrative management.

12.1 User Collection

The User collection stores the information required for user registration, authentication, and role-based access.

12.2 Category Collection

The Category collection stores the categories used to organize FAQ information. Each category contains a name and description and can be associated with multiple FAQ records.

12.3 FAQ Collection

The FAQ collection stores the questions and answers available to users. Each FAQ can be associated with a category and can contain keywords that are used by the search and AI queryprocessing functionality.

12.4 Database Relationship

The logical relationship between the main collections can be represented as:

Category → FAQ

One category can contain multiple FAQ records, while each FAQ is associated with a category.

The User collection is used independently for authentication and administrative access, while the FAQ and Category collections provide the main academic information used by the application.

12.5 Data Management

Mongoose schemas provide validation and structure for the stored data. The application performs create, read, update, and delete operations through the corresponding controllers and protected API routes.

The database design therefore provides a structured foundation for managing users, FAQ categories, and FAQ information within the AI FAQ Assistant.

13. USER ENTITY

The User Entity stores the details of registered users in the AI FAQ Assistant system. It contains user information such as name, email, password, role, and account creation date.

Main Fields:

- **_id** – Unique identifier for each user.
- **name** – Stores the user's name.
- **email** – Stores the unique email address.
- **password** – Stores the encrypted password.
- **role** – Defines the user as user or admin.
- **createdAt** – Stores the account creation date.

The User Entity is mainly used for authentication, authorization, and user management.

14. FAQ ENTITY

The FAQ Entity stores frequently asked questions and their answers in the system. It helps users quickly find academic information through normal FAQ search or the AI Assistant.

Main Fields:

- **_id** – Unique identifier for each FAQ.
- **question** – Stores the frequently asked question.
- **answer** – Stores the answer.
- **category** – Connects the FAQ with a category.
- **keywords** – Stores keywords related to the question.
- **views** – Tracks the number of views.
- **createdAt** – Stores the creation date.
- **updatedAt** – Stores the last update date.

The FAQ Entity is also used by the AI query processing engine to find the most relevant answer.

15. CATEGORY ENTITY

The Category Entity organizes FAQs into different academic and support categories. This makes it easier for users to find relevant information.

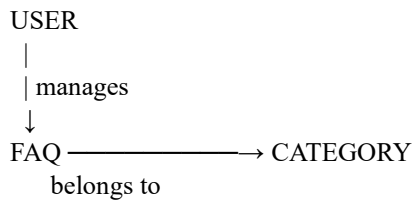
Main Fields:

- **_id** – Unique identifier for the category.
- **name** – Stores the category name.
- **description** – Provides details about the category.
- **createdAt** – Stores the category creation date.

The system includes categories such as General, College, Courses, Exams, Fees, Admission, Placements, Scholarships, and Technical Support.

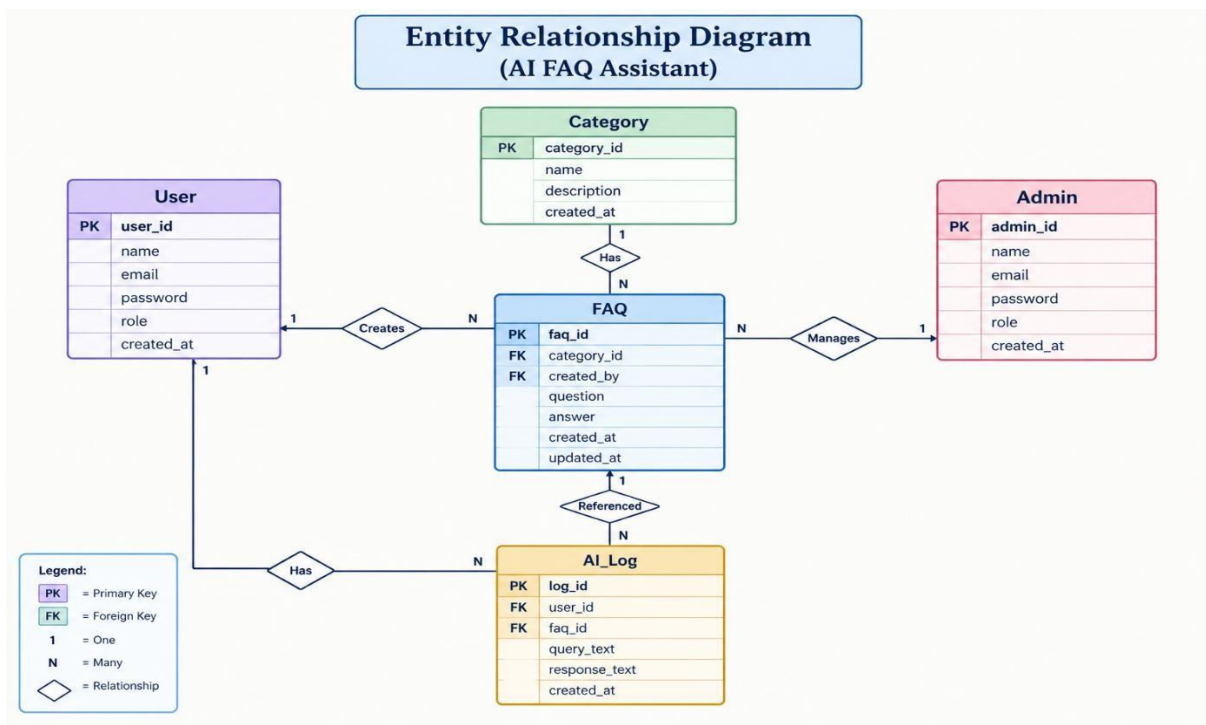
16. ENTITY RELATIONSHIP DIAGRAM

The AI FAQ Assistant system contains three main entities: User, FAQ, and Category.



- A User can access the FAQ system based on their role.
- Each FAQ belongs to one Category.
- A Category can contain multiple FAQs.

This relationship helps the system organize and retrieve FAQ information efficiently.



17. USER ROLES AND RESPONSIBILITIES

The AI FAQ Assistant system provides two main user roles: **Admin** and **User**. Each role has specific responsibilities and access permissions within the application.

17.1 Admin

The Admin is responsible for managing the overall system. The Admin can manage users, FAQs, and categories. Admin users can create, update, and delete FAQ records and categories. They can also view user information, check user statistics, delete users when required, and update user roles.

17.2 User

The User can register and log in to the application to access academic information. Users can view FAQs, search for questions, filter FAQs based on categories, and use the AI Assistant to find relevant answers. Regular users do not have permission to modify system data.

The separation of Admin and User responsibilities helps maintain proper control and secure management of the application.

18. ROLE-BASED ACCESS CONTROL

Role-Based Access Control (RBAC) is used to control the permissions of users based on their assigned role. In the AI FAQ Assistant, two roles are available: Admin and User.

Admin Access:

- Manage users and user roles.
- Create, update, and delete FAQs.
- Create, update, and delete categories.
- View user statistics.

User Access:

- Register and log in.
- View and search FAQs.
- Filter FAQs by category.
- Use the AI Assistant to get relevant answers.

The RBAC mechanism prevents regular users from accessing administrative operations and helps maintain secure system management.

19. AUTHENTICATION AND JWT

Authentication is used to verify the identity of users before they access protected features of the AI FAQ Assistant. The system uses JSON Web Token (JWT) for secure user authentication.

When a user logs in with a valid email and password, the server generates a JWT token. This token is stored on the client side and sent with requests to protected API routes.

The authentication process includes:

1. User enters email and password.
2. Server verifies the user credentials.
3. A JWT token is generated after successful login.
4. The token is used to access protected resources.
5. Admin-only operations are checked using the user's role.

JWT authentication helps maintain secure access to user and administrative functions in the system.

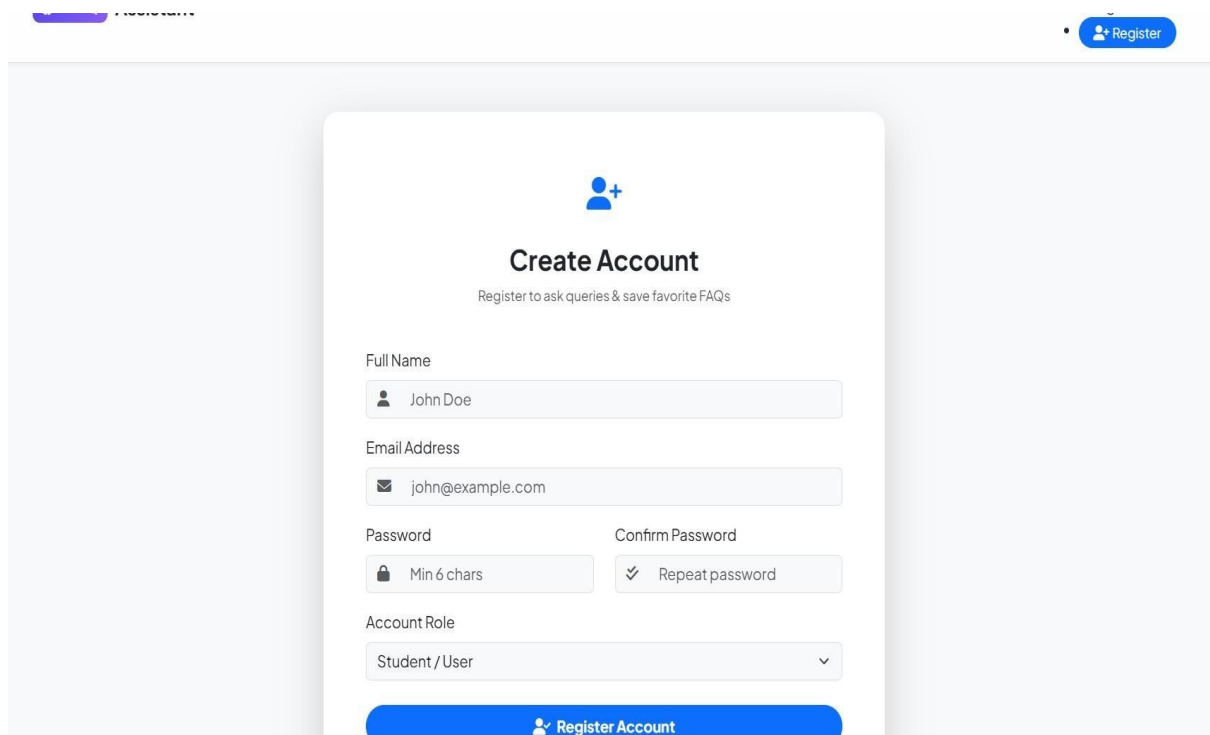
The image shows a user registration interface. At the top right, there is a blue 'Register' button with a user icon. The main form is titled 'Create Account' with a subtitle 'Register to ask queries & save favorite FAQs'. It contains several input fields: 'Full Name' with the value 'John Doe', 'Email Address' with the value 'john@example.com', 'Password' with a hint 'Min 6 chars', and 'Confirm Password' with a hint 'Repeat password'. There is also a dropdown for 'Account Role' currently set to 'Student / User'. At the bottom of the form is a blue 'Register Account' button with a checkmark icon.

Figure 4: User Registration Interface

20. FAQ MANAGEMENT

The FAQ Management module is used to maintain frequently asked questions and answers in the system. It allows the Admin to manage FAQ information, while Users can view and search the available FAQs.

Main Operations:

- **Create FAQ** – Admin can add a new question, answer, category, and keywords.
- **View FAQ** – Users can view available questions and answers.
- **Search FAQ** – Users can search for FAQs using relevant words or questions.
- **Update FAQ** – Admin can modify existing FAQ information.
- **Delete FAQ** – Admin can remove outdated or unnecessary FAQs.
- **Category Filtering** – FAQs can be filtered according to their categories.

This module keeps academic information organized and makes it easier for users to find the required answers.

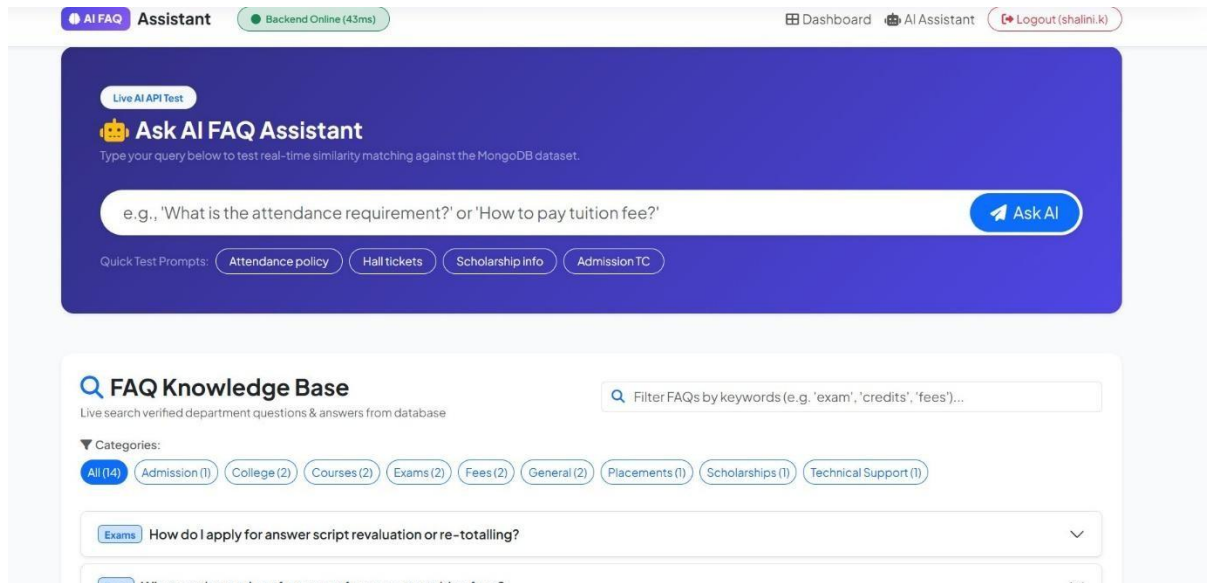


Figure 5: FAQ Management and AI Query Interface

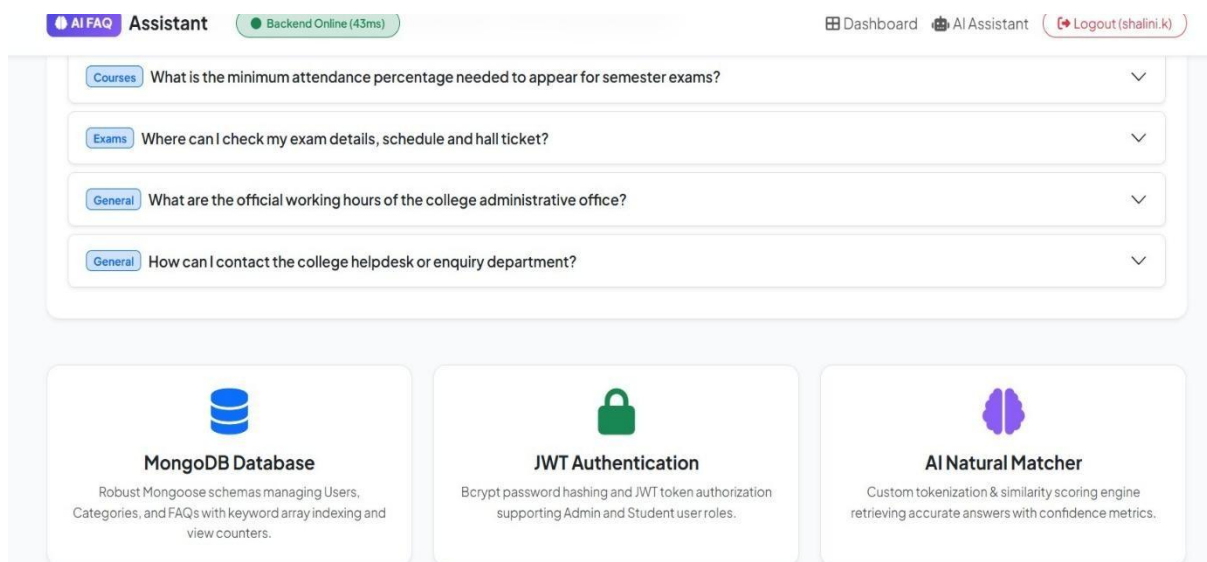


Figure 6: FAQ Knowledge Base and System Features

JS faqController.js X

```

nan mu-p > src > controllers > JS faqController.js > ...
1  const FAQ = require('../models/FAQ');
2  const Category = require('../models/Category');
3
4  // @desc    Get all FAQs (with optional category & search filter)
5  // @route    GET /api/faqs
6  // @access   Public
7  const getFAQs = async (req, res, next) => {
8    try {
9      const { category, search } = req.query;
10     let query = {};
11
12     if (category) {
13       query.category = category;
14     }
15
16     if (search && search.trim()) {
17       const searchRegex = new RegExp(search.trim(), 'i');
18       query.$or = [
19         { question: searchRegex },
20         { answer: searchRegex },
21         { keywords: searchRegex }
22       ];
23     }
24
25     const faqs = await FAQ.find(query)
26       .populate('category', 'name description')
27       .sort({ createdAt: -1 });
28
29     res.status(200).json({
30       success: true,
31       count: faqs.length,
32       data: faqs
33     });
34   } catch (error) {
35     next(error);
36   }
37 };
38
39 // @desc    Get single FAQ by ID
40 // @route    GET /api/faqs/:id
41 // @access   Public
42 const getFAQById = async (req, res, next) => {
43   try {
44     const faq = await FAQ.findById(req.params.id).populate('category', 'name description');
45     if (!faq) {
46       return res.status(404).json({
47         success: false,
48         message: 'FAQ not found'
49       });
50     }
51
52     // Increment view count
53     faq.views = (faq.views || 0) + 1;
54     await faq.save();
55
56     res.status(200).json({
57       success: true,
58       data: faq
59     });
60   } catch (error) {
61     next(error);
62   }
63 };
64
65 // @desc    Create FAQ
66 // @route    POST /api/faqs
67 // @access   Private/Admin
68 const createFAQ = async (req, res, next) => {
69   try {
70     const { question, answer, category, keywords } = req.body;
71
72     if (!question || !answer || !category) {
73       return res.status(400).json({
74         success: false,
75         message: 'Question, Answer, and Category are required'
76       });
77     }
78
79     // Verify category exists
80     const categoryExists = await Category.findById(category);
81     if (!categoryExists) {
82       return res.status(404).json({
83         success: false,
84         message: 'Selected Category does not exist'
85       });
86     }
87
88     // Parse keywords array
89     let keywordArray = [];
90     if (Array.isArray(keywords)) {
91       keywordArray = keywords.map(k => k.trim()).filter(Boolean);
92     } else if (typeof keywords === 'string') {
93       keywordArray = keywords.split(',').map(k => k.trim()).filter(Boolean);
94     }
95
96     const faq = await FAQ.create({
97       question: question.trim(),
98       answer: answer.trim(),
99       category,
100       keywords: keywordArray
101     });
102
103     const populatedFaq = await FAQ.findById(faq._id).populate('category', 'name description');
104
105     res.status(201).json({
106       success: true,
107       message: 'FAQ created successfully',
108       data: populatedFaq
109     });
110   } catch (error) {
111     next(error);
112   }
113 };
114
115 // @desc    Update FAQ
116 // @route    PUT /api/faqs/:id
117 // @access   Private/Admin
118 const updateFAQ = async (req, res, next) => {
119   try {
120     const { question, answer, category, keywords } = req.body;
121
122     const faq = await FAQ.findById(req.params.id);
123     if (!faq) {
124       return res.status(404).json({
125         success: false,
126         message: 'FAQ not found'
127       });
128     }
129
130     if (category) {
131       const categoryExists = await Category.findById(category);
132       if (!categoryExists) {

```

```

133         success: false,
134         message: 'Selected Category does not exist'
135     });
136     }
137     faq.category = category;
138 }
139 if (question !== undefined) faq.question = question.trim();
140 if (answer !== undefined) faq.answer = answer.trim();
141
142 if (keywords !== undefined) {
143     if (Array.isArray(keywords)) {
144         faq.keywords = keywords.map(k => k.trim()).filter(Boolean);
145     } else if (typeof keywords === 'string') {
146         faq.keywords = keywords.split(',').map(k => k.trim()).filter(Boolean);
147     }
148 }
149 faq.updatedAt = Date.now();
150 await faq.save();
151
152 const updatedFaq = await FAQ.findById(faq._id).populate('category', 'name description');
153
154 res.status(200).json({
155     success: true,
156     message: 'FAQ updated successfully',
157     data: updatedFaq
158 });
159 } catch (error) {
160     next(error);
161 }
162 };
163
164 // @desc Delete FAQ
165 // @route DELETE /api/faqs/:id
166 // @access Private/Admin
167 const deleteFAQ = async (req, res, next) => {
168     try {
169         const faq = await FAQ.findById(req.params.id);
170         if (!faq) {
171             return res.status(404).json({
172                 success: false,
173                 message: 'FAQ not found'
174             });
175         }
176
177         await faq.deleteOne();
178
179         res.status(200).json({
180             success: true,
181             message: 'FAQ deleted successfully'
182         });
183     } catch (error) {
184         next(error);
185     }
186 };
187
188 module.exports = {
189     getFAQs,
190     getFAQById,
191     createFAQ,
192     updateFAQ,
193     deleteFAQ
194 };
195

```

Figure 7: FAQ Management Controller Implementation

21. CATEGORY MANAGEMENT

The Category Management module is used to organize FAQs into different categories. It helps users find relevant academic information easily.

The system provides categories such as General, College, Courses, Exams, Fees, Admission, Placements, Scholarships, and Technical Support.

Main Operations:

- **View Categories** – Displays all available categories.
- **Create Category** – Admin can add a new category.
- **Update Category** – Admin can modify category details.
- **Delete Category** – Admin can remove a category when required.
- **FAQ Count** – The system displays the number of FAQs available under each category.

Category management keeps the FAQ data organized and improves the overall usability of the application.

22. AI QUERY PROCESSING

The AI Query Processing module helps users find relevant answers from the available FAQs. The system uses a custom keyword and token similarity-based processing engine instead of an external AI service.

When a user enters a question, the system processes the query by removing unnecessary words and identifying important keywords. It compares the query with stored FAQ questions, keywords, and answers to calculate a matching score.

If a suitable FAQ is found, the system returns the relevant answer along with a confidence percentage and related FAQ suggestions. If no suitable match is found, the system provides a no-match response and suggests available categories.

This module makes FAQ searching more intelligent and helps users receive relevant academic information quickly.

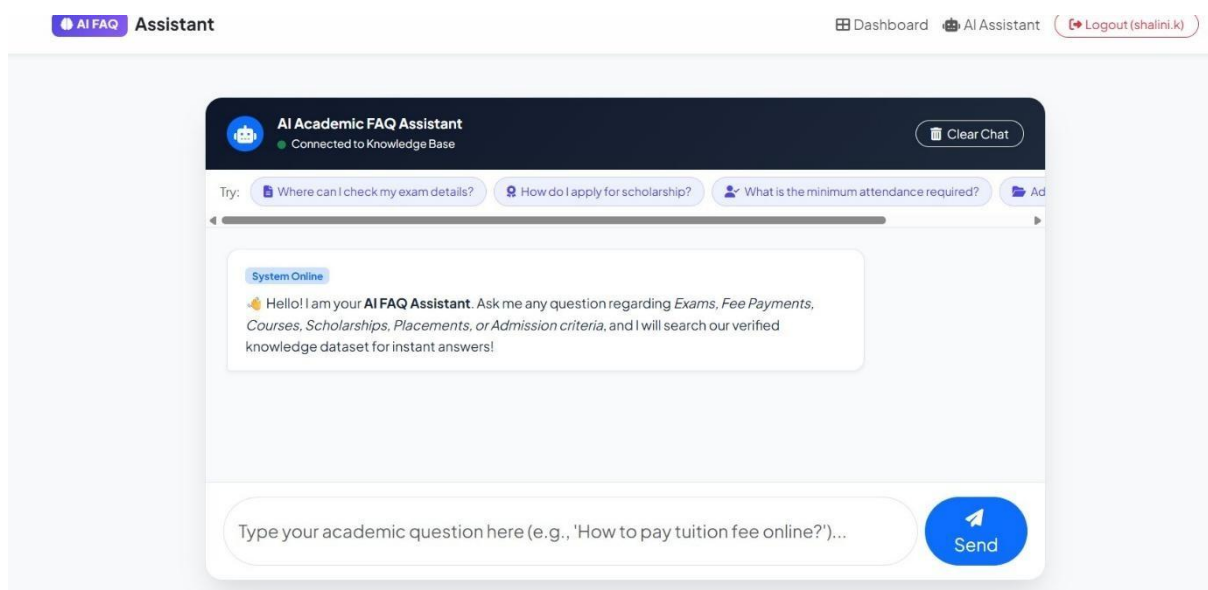


Figure 8: AI Academic FAQ Assistant Interface


```

JS faqController.js JS aiEngine.js X
nan mu-p > src > utils > JS aiEngine.js > processAIQuery
1  const STOP_WORDS = new Set([
2    'a', 'about', 'above', 'after', 'against', 'all', 'am', 'an', 'and', 'any', 'are', 'aren't',
3    'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by',
4    'can', 'can't', 'cannot', 'could', 'did', 'do', 'does', 'doing', 'down', 'during', 'each', 'few',
5    'for', 'from', 'further', 'had', 'has', 'have', 'having', 'he', 'her', 'here', 'hers', 'herself',
6    'him', 'himself', 'his', 'how', 'i', 'if', 'in', 'into', 'is', 'it', 'its', 'itself', 'just', 'me',
7    'more', 'most', 'my', 'myself', 'no', 'nor', 'not', 'now', 'of', 'off', 'on', 'once', 'only', 'or',
8    'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 'same', 'should', 'so', 'some', 'such',
9    'than', 'that', 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there', 'these', 'they',
10   'this', 'those', 'through', 'to', 'too', 'under', 'until', 'up', 'very', 'was', 'we', 'were', 'what',
11   'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'with', 'would', 'you', 'your', 'yours'
12  ]);
13  function tokenize(text) {
14    if (!text) return [];
15    return text
16      .toLowerCase()
17      .replace(/[^\w\s]/g, '')
18      .split(/\s+/)
19      .filter(word => word.length > 1 && !STOP_WORDS.has(word));
20  }
21
22  function computeSimilarity(userQuery, faq) {
23    const queryTokens = tokenize(userQuery);
24    if (queryTokens.length === 0) return { score: 0, faq };
25    const rawQuery = userQuery.toLowerCase().trim();
26    const rawQuestion = faq.question.toLowerCase().trim();
27    const rawAnswer = faq.answer.toLowerCase().trim();
28    const rawKeywords = Array.isArray(faq.keywords)
29      ? faq.keywords.map(k => k.toLowerCase().trim())
30      : (faq.keywords || []).toLowerCase().split(',').map(k => k.trim());
31
32    let score = 0;
33    // 1. Exact or near-exact question match
34    if (rawQuestion === rawQuery) {
35      score = 150;
36    } else if (rawQuestion.includes(rawQuery) || rawQuery.includes(rawQuestion)) {
37      score += 90;
38    }
39
40    // 2. Keyword exact / token matches
41    rawKeywords.forEach(kw => {
42      if (!kw) return;
43      if (rawQuery.includes(kw)) {
44        score += 40;
45      }
46      const kwTokens = tokenize(kw);
47      kwTokens.forEach(token => {
48        if (queryTokens.includes(token)) {
49          score += 25;
50        }
51      });
52    });
53
54    // 3. Question Token Match
55    const questionTokens = tokenize(faq.question);
56    let matchedTokensCount = 0;
57    queryTokens.forEach(token => {
58      if (questionTokens.includes(token)) {
59        matchedTokensCount++;
60        score += 25;
61      }
62    });
63
64    // 4. Answer Token Match
65    const answerTokens = tokenize(faq.answer);
66    queryTokens.forEach(token => {
67      if (answerTokens.includes(token)) {
68        score += 8;
69      }
70    });
71
72    // Coverage bonus: ratio of query tokens matched in question
73    const coverageRatio = matchedTokensCount / queryTokens.length;
74    score += Math.round(coverageRatio * 30);
75
76    return { score, faq };
77  }
78
79  function processAIQuery(userQuery, faqs) {
80    if (userQuery || typeof userQuery !== 'string' || !userQuery.trim()) {
81      return {
82        found: false,
83        message: "Please enter a valid question or query."
84      };
85    }
86    if (!faqs || faqs.length === 0) {
87      return {
88        found: false,
89        message: "No FAQ data is currently available in the knowledge base."
90      };
91    }
92    const scoredFaqs = faqs.map(faq => computeSimilarity(userQuery, faq));
93    // Sort descending by score
94    scoredFaqs.sort((a, b) => b.score - a.score);
95    const topMatch = scoredFaqs[0];
96    const threshold = 18; // Minimum threshold required for high-confidence match
97    if (topMatch && topMatch.score >= threshold) {
98      // Get related suggestions (top 2 next matches if score > 18)
99      const related = scoredFaqs
100        .slice(1, 4)
101        .filter(item => item.score >= 12)
102        .map(item => ({
103          id: item.faq_id,
104          question: item.faq.question,
105          category: item.faq.category
106        }));
107      // Confidence calculation (capped at 98%)
108      const confidence = Math.min(98, Math.max(60, Math.round(topMatch.score * 0.8)));
109      return {
110        found: true,
111        faq: topMatch.faq,
112        confidence: `${confidence}%`,
113        related: related,
114        matchScore: topMatch.score
115      };
116    }
117    return {
118      found: false,
119      message: "Sorry, I couldn't find an exact or relevant FAQ answer matching your question in the system database. Please rephrase your query or select a category to browse related topics.",
120      suggestedCategories: Array.from(new Set(faqs.map(f => f.category?.name).filter(Boolean))).slice(0, 5)
121    };
122  }
123
124  module.exports = {
125    tokenize,
126    computeSimilarity,
127    processAIQuery
128  };

```

Figure 9: AI Query Processing and Similarity Matching

23. AI QUERY MATCHING PROCESS

The AI Query Matching Process compares the user's question with the available FAQ data to identify the most relevant answer.

The process includes:

1. The user enters a question in the AI Assistant.
2. The system validates the query.
3. The query is converted into tokens and unnecessary words are removed.
4. The processed query is compared with FAQ questions, keywords, and answers.
5. A similarity score is calculated for each FAQ.
6. FAQs are arranged based on their matching score.
7. If the score reaches the required threshold, the most relevant FAQ answer is returned.
8. Related FAQ suggestions are also provided when available.

This process allows the system to provide relevant answers without depending on an external AI API.

24. AI CONFIDENCE AND RELATED SUGGESTIONS

The AI Assistant provides a confidence percentage along with the matched FAQ answer. The confidence value is calculated from the matching score obtained during query processing.

When a suitable FAQ is identified, the system returns:

- **Matched FAQ** – The most relevant question and answer.
- **Confidence** – Indicates the matching level of the selected FAQ.
- **Related FAQs** – Provides additional relevant questions based on their scores.
- **Match Score** – Represents the calculated similarity between the user's query and FAQ data.

If no suitable FAQ is found, the system displays a no-match message and provides available category suggestions. This helps users continue searching for relevant academic information.

25. SECURITY FEATURES

The AI FAQ Assistant includes several security features to protect user accounts, system data, and administrative operations. These features ensure that only authorized users can access protected resources and perform management operations.

25.1 Password Protection

User passwords are securely hashed using bcryptjs before they are stored in the database. The original password is not stored directly.

25.2 JWT Authentication

The system uses JSON Web Token (JWT) for user authentication. After successful login, a token is generated and used to access protected API routes.

25.3 Role-Based Authorization

The system separates users into User and Admin roles. Administrative operations such as managing FAQs, categories, and users are restricted to Admin users.

25.4 Input Validation

The application validates important user inputs such as email, password, FAQ questions, and required fields. This helps prevent invalid data from being stored in the system.

25.5 Protected API Routes

Sensitive API operations are protected using authentication middleware. Requests without valid authentication cannot access protected resources.

These security mechanisms help maintain the confidentiality and integrity of user accounts and application data.

26. API MODULE SUMMARY

The AI FAQ Assistant is developed using RESTful APIs to connect the frontend application with the backend server. The APIs handle authentication, FAQ management, category management, AI query processing, and user administration.

26.1 Authentication API

Handles user registration, login, and retrieval of the currently authenticated user.

26.2 Category API

Provides operations to view, create, update, and delete FAQ categories. Create, update, and delete operations are restricted to Admin users.

26.3 FAQ API

Handles the creation, retrieval, updating, deletion, searching, and category-based filtering of FAQs.

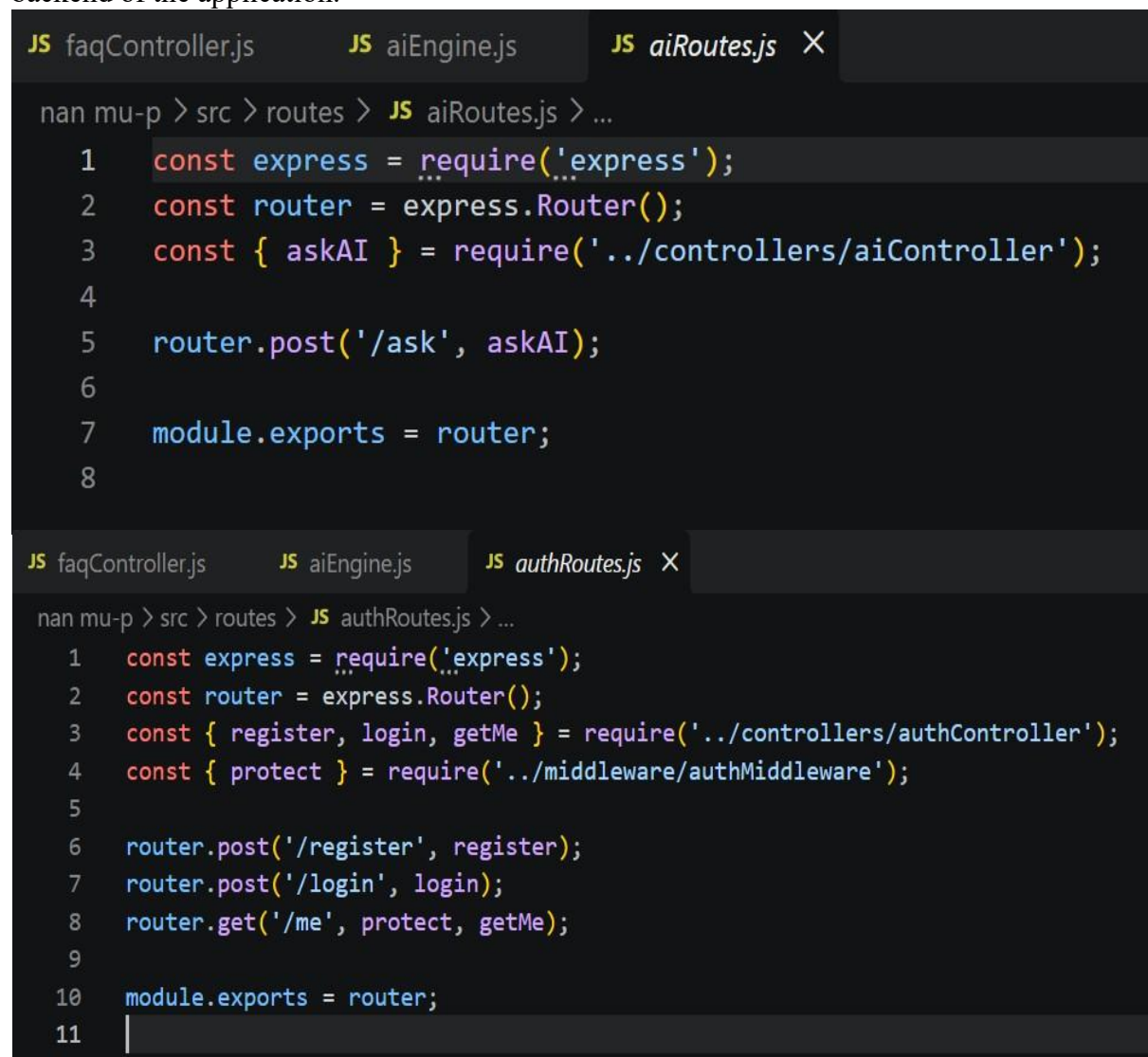
26.4 AI API

The AI API receives a user's question through the `/api/ai/ask` endpoint and processes it using the custom AI query matching engine. It returns the most relevant FAQ, confidence value, and related suggestions.

26.5 User API

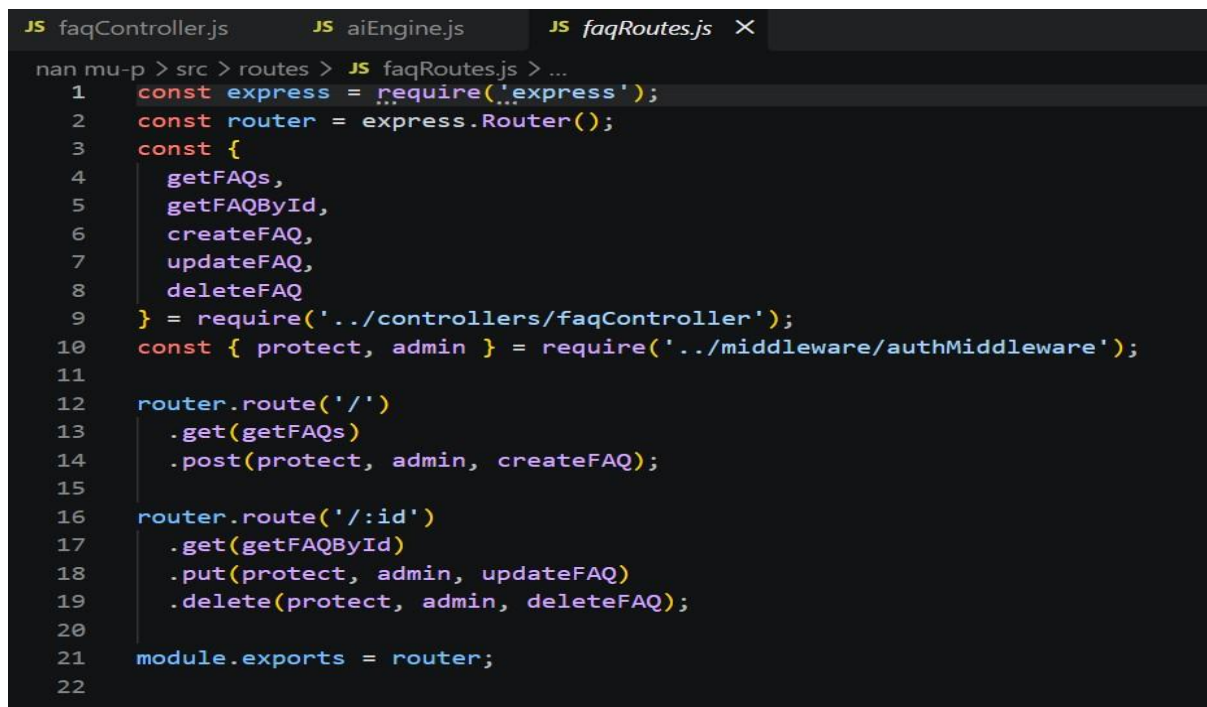
Provides Admin-level operations for viewing users, checking user statistics, deleting users, and updating user roles.

These API modules provide a structured communication layer between the frontend and backend of the application.



```
JS faqController.js    JS aiEngine.js    JS aiRoutes.js X
nan mu-p > src > routes > JS aiRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const { askAI } = require('../controllers/aiController');
4
5  router.post('/ask', askAI);
6
7  module.exports = router;
8

JS faqController.js    JS aiEngine.js    JS authRoutes.js X
nan mu-p > src > routes > JS authRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const { register, login, getMe } = require('../controllers/authController');
4  const { protect } = require('../middleware/authMiddleware');
5
6  router.post('/register', register);
7  router.post('/login', login);
8  router.get('/me', protect, getMe);
9
10 module.exports = router;
11
```

The image shows a code editor with three tabs: 'faqController.js', 'aiEngine.js', and 'faqRoutes.js'. The 'faqRoutes.js' tab is active, displaying the following JavaScript code:

```
nan mu-p > src > routes > JS faqRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    getFAQs,
5    getFAQById,
6    createFAQ,
7    updateFAQ,
8    deleteFAQ
9  } = require('../controllers/faqController');
10 const { protect, admin } = require('../middleware/authMiddleware');
11
12 router.route('/')
13   .get(getFAQs)
14   .post(protect, admin, createFAQ);
15
16 router.route('/:id')
17   .get(getFAQById)
18   .put(protect, admin, updateFAQ)
19   .delete(protect, admin, deleteFAQ);
20
21 module.exports = router;
22
```

Figure 10: API Route Implementation

27. API TESTING

API testing is performed to verify that the backend APIs of the AI FAQ Assistant work correctly and return the expected responses. The APIs can be tested using tools such as Postman.

The main API operations tested include:

- **User Registration** – Checks whether a new user can be registered with valid details.
- **User Login** – Verifies email and password and returns a JWT token for successful authentication.
- **FAQ Operations** – Tests creating, viewing, updating, deleting, searching, and filtering FAQ records.
- **Category Operations** – Checks category creation, retrieval, updating, and deletion.
- **AI Query** – Sends a user question to the AI API and verifies whether a relevant FAQ answer is returned.
- **User Management** – Tests Admin operations such as viewing users, checking statistics, deleting users, and updating user roles.
- **Protected Routes** – Verifies that restricted operations require valid authentication and appropriate Admin access.

API testing helps identify request, authentication, validation, and response-related errors before the application is used by end users.

28. TESTING RESULTS

The testing process was carried out to verify the major functions of the AI FAQ Assistant. The application was checked for authentication, FAQ management, category management, AI query processing, and user management operations.

Test Module	Expected Result	Result
User Registration	User account should be created successfully	Pass
User Login	Valid credentials should generate JWT token	Pass
FAQ Management	Admin should be able to manage FAQs	Pass
Category Management	Admin should be able to manage categories	Pass
FAQ Search	Relevant FAQs should be displayed	Pass
AI Query Processing	Relevant answer should be returned for a matching query	Pass
User Management	Admin should be able to manage users	Pass
Protected Routes	Unauthorized access should be restricted	Pass

The testing results show that the major modules of the application perform their intended functions and that the main user and Admin operations work correctly.

29. MONGODB CONNECTION

MongoDB is used as the database for storing and managing the data of the AI FAQ Assistant. The application connects to MongoDB using Mongoose, which provides a structured way to work with MongoDB collections and documents.

The database connection is configured using the MONGODB_URI environment variable. The default local database is:

```
mongodb://127.0.0.1:27017/ai_faq_assistant
```

The system stores important application data such as Users, Categories, and FAQs in MongoDB. Mongoose models are used to define the structure and validation rules for these collections.

The application also includes a fallback mechanism using an in-memory MongoDB server when the primary MongoDB connection is unavailable. This helps the application continue running during development and testing.

```
JS faqController.js JS aiEngine.js db.js X
nan mu-p > src > config > JS db.js > ...
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    const uri = process.env.MONGODB_URI || 'mongodb://127.0.0.1:27017/ai_faq_assistant';
5
6    try {
7      // Set connection timeout options
8      const conn = await mongoose.connect(uri, {
9        serverSelectionTimeoutMS: 3000
10      });
11      console.log(`[MongoDB] Connected to host: ${conn.connection.host}`);
12      return conn;
13    } catch (err) {
14      console.warn(`[MongoDB] Primary MongoDB connection failed (${err.message}).`);
15      console.log(`[MongoDB] Starting In-Memory MongoDB Server fallback for zero-config demonstration...`);
16
17      try {
18        const { MongoMemoryServer } = require('mongodb-memory-server');
19        const mongoServer = await MongoMemoryServer.create();
20        const memoryUri = mongoServer.getUri();
21        const conn = await mongoose.connect(memoryUri);
22        console.log(`[MongoDB] Connected to In-Memory MongoDB at: ${memoryUri}`);
23        return conn;
24      } catch (memErr) {
25        console.error(`[MongoDB] Memory server connection error:`, memErr.message);
26        process.exit(1);
27      }
28    }
29  };
30
31  module.exports = connectDB;
32
```

Figure 11: MongoDB Database Connection

30. SYSTEM FUNCTIONALITY SUMMARY

The AI FAQ Assistant is a web-based application designed to help users find academic information through FAQs and an AI-based query processing system. The application provides separate features for normal users and administrators.

The major functionalities of the system are:

- **User Authentication:** Users can register, log in, and access protected features using JWT authentication.
- **FAQ Management:** FAQs can be created, viewed, searched, updated, and deleted based on user permissions.
- **Category Management:** FAQs are organized into categories such as Courses, Exams, Admission, Fees, and Placements.
- **AI Query Processing:** User questions are processed using the custom keyword and token similarity engine to identify relevant FAQ answers.
- **Admin Management:** Admin users can manage FAQs, categories, users, and user roles.
- **Secure Access:** Authentication middleware and role-based authorization protect administrative operations.
- **Search and Filtering:** Users can search FAQs and filter them according to categories.

Overall, these functions work together to provide an organized and user-friendly platform for accessing academic information and obtaining relevant answers through the AI Assistant.

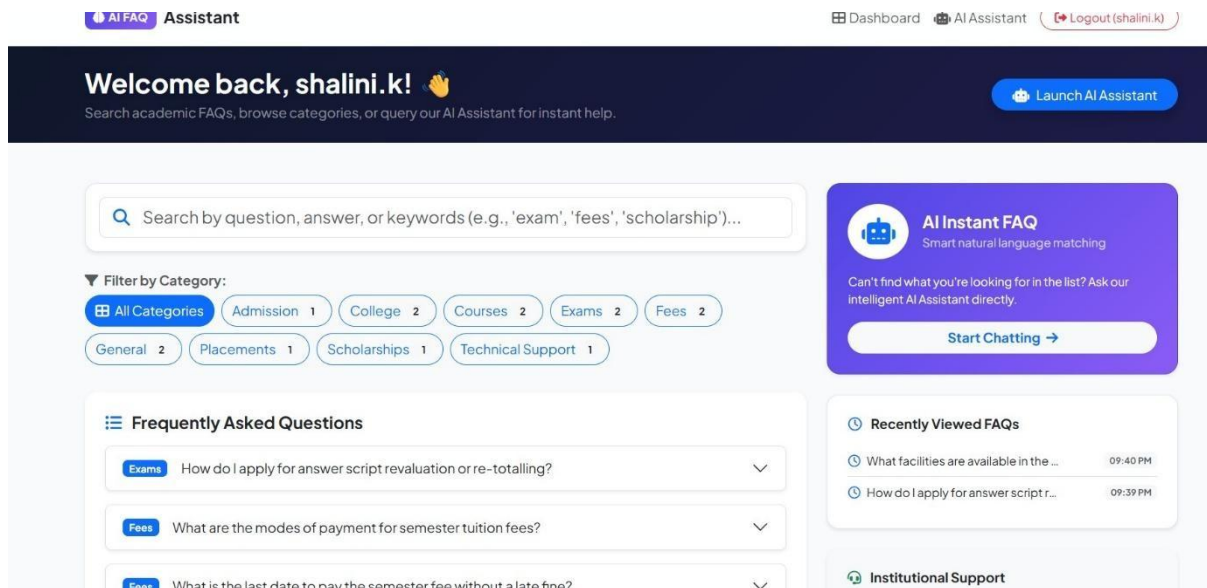


Figure 12: Student Dashboard

31. ADVANTAGES

The AI FAQ Assistant provides several advantages for users and administrators by making academic information easier to access, manage, and retrieve.

31.1 Easy Access to Information

Users can access frequently asked academic questions and answers through a simple web interface without manually searching through large amounts of information.

31.2 Faster Answer Retrieval

The AI query processing engine compares the user's question with available FAQ data and provides a relevant answer quickly.

31.3 Organized FAQ Management

FAQs are arranged into categories such as Courses, Exams, Admission, Fees, Placements, and Scholarships, making information easier to locate.

31.4 Secure User Access

JWT authentication and role-based access control help protect user accounts and restrict administrative functions to authorized Admin users.

31.5 Centralized Data Management

User, FAQ, and category information is stored in MongoDB, providing a centralized database for managing application data.

31.6 Related FAQ Suggestions

When a suitable answer is found, the system can provide related FAQ suggestions, helping users explore additional relevant information.

Overall, the system reduces the effort required to find academic information while providing administrators with effective tools to maintain the FAQ database.

32. LIMITATIONS

The AI FAQ Assistant provides useful features for academic information retrieval, but the system has some limitations.

32.1 FAQ-Based Knowledge

The AI Assistant provides answers based on the FAQs stored in the database. If the required information is not available in the FAQ data, the system may not be able to provide the expected answer.

32.2 Keyword Matching Dependency

The AI query processing system uses keyword and token similarity to identify relevant FAQs. Therefore, questions with very different wording may sometimes produce a lower matching score.

32.3 Limited AI Processing

The system uses a custom query matching engine rather than an external generative AI service. Therefore, it does not generate completely new answers beyond the available FAQ information.

32.4 Database Dependency

The availability of FAQ information depends on the data stored in the MongoDB database. Incorrect or outdated FAQ data can affect the relevance of the responses.

32.5 Internet and Server Dependency

Users need access to the web application and its backend server to use the system. Server or database connection issues may affect application functionality.

These limitations can be addressed through future improvements to make the system more flexible and capable.

33. FUTURE ENHANCEMENTS

The AI FAQ Assistant can be further improved by adding additional features and enhancing the existing query processing capabilities.

33.1 Improved AI Query Understanding

The query processing system can be enhanced to understand different forms of questions and provide more accurate FAQ matches.

33.2 Larger FAQ Knowledge Base

More academic FAQs can be added to cover additional subjects, courses, examinations, scholarships, placements, and student services.

33.3 Advanced Search

The system can be extended with more advanced search techniques to improve the retrieval of relevant information from a larger FAQ database.

33.4 Improved User Interface

The frontend can be enhanced with additional interactive features and a more responsive design for better usability across different devices.

33.5 Analytics and Reports

Future versions can include detailed usage reports, such as frequently searched questions and commonly accessed FAQ categories.

These enhancements can make the system more flexible, informative, and useful for students in future versions.

34. CONCLUSION

The AI FAQ Assistant is a web-based academic information system developed to help users find frequently asked questions and relevant answers in an organized manner. The system provides user registration, login, FAQ search, category-based filtering, and an AI Assistant for processing user queries.

The application uses Node.js, Express.js, MongoDB, Mongoose, JWT, bcryptjs, HTML, CSS, and JavaScript to provide its main functionalities. The custom AI query processing engine compares user queries with stored FAQ information using keyword and token similarity to identify relevant answers.

The system also provides Admin features for managing users, categories, and FAQs while maintaining secure access through authentication and role-based authorization. Although the current system has some limitations, it provides a useful foundation for academic information retrieval and can be enhanced with more advanced query processing and additional features in the future.

API TESTING VIDEO:

https://drive.google.com/file/d/1MJZC4UNVnb2FWG10hPWJoMMm9Da_T4xk/view?usp=drive_link

DEMOLINK:

https://drive.google.com/file/d/1kJHAzvzYbXafnxhh1JC79nRTt1Pi6JEL/view?usp=drive_link